Pooling

1. Code:

```
import numpy as np
from .resampling import *


class MaxPool2d_stride1:
    def __init__(self, kernel):
        self.K = kernel

    def forward(self, x):
        self.x = x
        N, C_in, H_in, W_in = x.shape
        C_out = C_in  # max pool does not modify the no. of channels
        H_out = (H_in - self.K) + 1
        W_out = (W_in - self.K) + 1
        z = np.empty(shape=(N, C_out, H_out, W_out), dtype=np.float64)
        for i in range(H_out):
            for j in range(W_out):
                z[:, :, i, j] = np.max(
                    x[:, :, i : i + self.K, j : j + self.K], axis=(2, 3)
                )

        # # save indices of maximum elements for backprop
        # max_xy = np.empty(shape=(N, C_out, H_out, W_out, 2))
        # self.max_x = np.empty(shape=(N, C_out, H_out, W_out), dtype=np.int64)
        # self.max_y = np.empty(shape=(N, C_out, H_out, W_out), dtype=np.int64)
        # for i in range(N):
        #     for j in range(H_out):
        #         for k in range(W_out):
        #             max_xy[i, :, j, k, :] = np.array([np.unravel_index(np.argmax(c), c.shape) for c in
        #                                               x[i, :, j:j + self.K, k:k + self.K]])
        #             self.max_x[i, :, j, k] = max_xy[i, :, j, k, 0] + j
        #             self.max_y[i, :, j, k] = max_xy[i, :, j, k, 1] + k
        return z

    def backward(self, dLdz):
        N, C_out, H_out, W_out = dLdz.shape
        dLdx = np.zeros(shape=self.x.shape, dtype=np.float64)

        # # option 1
        # for i in range(N):
        #     for j in range(C_out):
        #         for k in range(H_out):
        #             for l in range(W_out):
        #                 dLdx[i, j, self.max_x[i, j, k, l], self.max_y[i, j, k, l]] += dLdz[i, j, k, l]

        # option 2: more concise
        for i in range(H_out):
            for j in range(W_out):
                mask = self.x[:, :, i : i + self.K, j : j + self.K] == np.max(
                    self.x[:, :, i : i + self.K, j : j + self.K],
                    axis=(2, 3),
                    keepdims=True,
                )
                dLdx[:, :, i : i + self.K, j : j + self.K] += (
                    mask * (dLdz[:, :, i, j])[:, :, np.newaxis, np.newaxis]
                )

        return dLdx


class MeanPool2d_stride1:
    def __init__(self, kernel):
        self.K = kernel

    def forward(self, x):
        self.x = x
        N, C_in, H_in, W_in = x.shape
        C_out = C_in
        H_out = (H_in - self.K) + 1
        W_out = (W_in - self.K) + 1
        # forward
        z = np.empty(shape=(N, C_out, H_out, W_out), dtype=np.float64)
        for i in range(H_out):
            for j in range(W_out):
                z[:, :, i, j] = np.mean(
                    x[:, :, i : i + self.K, j : j + self.K], axis=(2, 3)
                )
        return z

    def backward(self, dLdz):
        # backward
        N, C_in, H_in, W_in = self.x.shape
        dLdx = np.zeros(shape=self.x.shape, dtype=np.float64)
```

```
        dLdz = np.pad(
            dLdz,
            pad_width=(
                (0, 0),
                (0, 0),
                (self.K - 1, self.K - 1),
                (self.K - 1, self.K - 1),
            ),
            mode="constant",
        )
        for i in range(H_in):
            for j in range(W_in):
                dLdx[:, :, i, j] += np.mean(
                    dLdz[:, :, i : i + self.K, j : j + self.K], axis=(2, 3)
                )
        return dLdx


class MaxPool2d:
    def __init__(self, kernel, stride):
        self.K = kernel
        self.stride = stride

        self.maxpool2d_stride1 = MaxPool2d_stride1(kernel)
        self.downsample2d = Downsample2d(stride)

    def forward(self, x):
        z = self.maxpool2d_stride1.forward(x)
        z = self.downsample2d.forward(z)
        return z

    def backward(self, dLdz):
        dLdx = self.downsample2d.backward(dLdz)
        dLdx = self.maxpool2d_stride1.backward(dLdx)
        return dLdx


class MeanPool2d:
    def __init__(self, kernel, stride):
        self.K = kernel
        self.stride = stride

        self.meanpool2d_stride1 = MeanPool2d_stride1(kernel)
        self.downsample2d = Downsample2d(stride)

    def forward(self, x):
        z = self.meanpool2d_stride1.forward(x)
        z = self.downsample2d.forward(z)
        return z

    def backward(self, dLdz):
        dLdx = self.downsample2d.backward(dLdz)
        dLdx = self.meanpool2d_stride1.backward(dLdx)
        return dLdx


if __name__ == "__main__":
    N = 2
    C_in = 3
    H_in = 5
    W_in = 5
    kernel_size = 3

    x = np.random.randint(0, 10, (N, C_in, H_in, W_in))
    maxpool_stride1 = MaxPool2d_stride1(kernel_size)
    z = maxpool_stride1.forward(x)

    dLdz = np.random.randint(0, 10, size=z.shape)
    dLdx = maxpool_stride1.backward(dLdz)
```